

UNIDAD I — FUNDAMENTOS Y DISEÑO DE SOLUCIONES DE INTELIGENCIA DE NEGOCIOS

LABORATORIO 1

Del dato crudo a la decisión gerencial

Diagnóstico de fuentes, definición única de métricas y construcción de indicadores

1. Datos generales

Laboratorio	Laboratorio 1: Del dato crudo a la decisión gerencial
Modalidad de trabajo	INDIVIDUAL. Cada estudiante desarrolla y entrega su propio trabajo.
Herramienta	Python 3.10 o superior con la librería pandas
Resultado de aprendizaje	CT 12.2 RA1 — Comprende los conceptos fundamentales de la Inteligencia de Negocios (BI)

2. Logro del laboratorio

Al finalizar el laboratorio, el estudiante diagnostica la calidad de un conjunto de fuentes de datos operacionales, demuestra empíricamente por qué una misma pregunta de negocio produce respuestas distintas cuando no existe una definición única de la métrica, propone e implementa esa definición única, y construye un conjunto de indicadores verificables contra criterios de calidad.

2.1. Objetivos específicos

- Cargar e inspeccionar cinco fuentes de datos heterogéneas y elaborar su diccionario de datos.
- Detectar y cuantificar diez problemas de calidad de datos de tipos distintos.
- Aplicar reglas de limpieza documentadas y trazables, dejando registro del efecto de cada una.
- Reproducir numéricamente el problema de las tres cifras presentado en la sesión teórica.
- Formular una definición única de venta neta y calcular un panel de seis indicadores.
- Clasificar cada indicador según el nivel de decisión que soporta.

3. Requisitos previos

Este laboratorio se apoya en las competencias desarrolladas en las asignaturas prerrequisito. Se espera que el estudiante domine lo siguiente:

- Base de Datos II: consultas de agregación y agrupamiento, combinaciones entre tablas, llaves primarias y foráneas, integridad referencial.
- Análisis y Diseño de Algoritmos: manejo de estructuras de datos y nociones básicas de complejidad.
- Estadística descriptiva: media, mediana, percentiles y detección de valores atípicos.

No se requiere experiencia previa con pandas. Todos los comandos necesarios se entregan en esta guía, y el Anexo B incluye una tabla de equivalencias entre las operaciones de pandas y sus equivalentes en SQL, que el estudiante ya conoce.

4. Contexto: Comercial Los Andes S.A.C.

Se trabaja sobre el mismo caso presentado en la sesión teórica. Comercial Los Andes S.A.C. es una cadena peruana de mejoramiento del hogar con doce tiendas en Lima, Arequipa y Trujillo, más un canal de venta en línea. Factura aproximadamente S/ 48 millones anuales.

En la reunión de resultados de agosto de 2026, tres áreas reportaron cifras distintas de venta para el mismo mes. La gerencia general encargó a un practicante de Ingeniería de Software —el rol que usted asume en este laboratorio— que determine el origen de la discrepancia y proponga una definición única.

Su encargo

1. Explicar por qué las tres áreas obtienen cifras distintas, con evidencia numérica y no con opiniones.
2. Diagnosticar el estado de las fuentes de datos e informar qué tan confiables son.
3. Proponer una definición única de «venta neta» y calcular el valor resultante.
4. Entregar un panel mínimo de indicadores que la gerencia pueda usar el próximo lunes.

5. Datos del laboratorio

5.1. Obtención de los archivos

Los datos están publicados en el Aula Virtual de la asignatura, en la carpeta de la Semana 1, bajo el nombre datos_lab01.zip. Descargue el archivo y descomprímalo. Los seis archivos son de formato CSV, con coma como delimitador, punto como separador decimal y codificación UTF-8.

Advertencia importante: los archivos contienen problemas de calidad de datos introducidos deliberadamente con fines pedagógicos. No son errores de la guía. Detectarlos y documentarlos es parte del trabajo.

5.2. Estructura de carpetas de trabajo

Cree la siguiente estructura antes de comenzar. Respetarla es obligatorio: el entregable se evalúa ejecutando su código, y las rutas relativas deben funcionar sin modificación.

```
lab01_<apellido>/
|
+-- datos/                                <- aqui van los 6 archivos CSV
|   +-- ventas_pos_202608.csv
|   +-- comprobantes_202608.csv
|   +-- despachos_202608.csv
|   +-- productos.csv
|   +-- tiendas.csv
|   +-- ventas_mensual_2025_2026.csv
|
+-- lab01.py                             <- su codigo
+-- salidas/                             <- resultados que genere su codigo
+-- informe_lab01.pdf                     <- su informe
```

5.3. Diccionario de datos

La tabla siguiente describe cada archivo. Note que la descripción es la que entregó el área de TI de la empresa: como ocurre en la realidad, es incompleta y en algunos puntos no coincide con lo que los datos realmente contienen. Verificarlo es parte de la Parte 2.

Archivo	Filas aprox.	Contenido	Origen
ventas_pos_202608.csv	34 900	Detalle de líneas de pedido de agosto 2026: pedido, fecha, tienda, canal, SKU, cantidad, precio unitario, descuento y estado del pedido.	Sistemas POS de las tiendas
comprobantes_202608.csv	12 700	Comprobantes electrónicos emitidos en agosto 2026: boletas, facturas y notas de crédito, con el monto total incluido el IGV.	Sistema ERP — Facturación
despachos_202608.csv	13 300	Guías de remisión con fecha de despacho, estado y valorización a precio de lista.	Sistema ERP — Logística
productos.csv	270	Catálogo de productos: SKU, nombre, categoría, marca, precio de lista y unidad de medida.	Sistema ERP — Maestros
tiendas.csv	13	Maestro de locales: código, nombre, ciudad, región, metros cuadrados de sala y fecha de apertura.	Sistema ERP — Maestros
ventas_mensual_2025_2026.csv	1 480	Agregado mensual por tienda y categoría desde enero 2025: unidades, venta neta, costo de mercadería, número de transacciones y de clientes. Se usa en la actividad propuesta.	Reportes históricos consolidados

Tabla 1. Archivos entregados para el laboratorio.

Detalle de columnas de ventas_pos_202608.csv

Columna	Tipo esperado	Descripción
id_linea	Texto	Identificador único de la línea de pedido
id_pedido	Texto	Identificador del pedido. Un pedido tiene una o más líneas
fecha_pedido	Fecha	Fecha de registro del pedido
codigo_tienda	Texto	Código del local. Se relaciona con tiendas.codigo_tienda
canal	Texto	TIENDA o DIGITAL
sku	Texto	Código del producto. Se relaciona con productos.sku
cantidad	Entero	Unidades vendidas en la línea
precio_unitario	Decimal	Precio de venta unitario, INCLUIDO el IGV
descuento_pct	Decimal	Descuento aplicado, expresado en tanto por uno (0.15 = 15%)
estado_pedido	Texto	REGISTRADO o ANULADO

Tabla 2. Columnas del archivo de detalle de ventas.

Regla de negocio que debe conocer

El precio_unitario del archivo de ventas y el monto_total de los comprobantes están expresados CON IGV. La tasa vigente del IGV en el Perú es 18%.

El valor de una línea de pedido se calcula como: cantidad × precio_unitario × (1 – descuento_pct).

Las notas de crédito se registran con monto positivo. Para obtener el neto facturado deben restarse.

Parte 0. Preparación del entorno

Tiempo estimado: 10 minutos.

Paso 0.1. Verificar la instalación de Python

Abra una terminal (Símbolo del sistema o PowerShell en Windows, Terminal en Linux o macOS) y ejecute:

```
python --version
```

Debe obtener la versión 3.10 o superior. Si el comando no se reconoce, pruebe con `python3 --version`. Si tampoco funciona, instale Python desde python.org marcando la casilla "Add Python to PATH" durante la instalación.

Paso 0.2. Instalar pandas

```
pip install pandas
```

Si ya está instalado, el comando lo indicará y no hará ningún cambio. Verifique la instalación con:

```
python -c "import pandas; print(pandas.__version__)"
```

Paso 0.3. Crear la estructura de trabajo

Cree la carpeta `lab01_<su apellido>` según el esquema de la sección 5.2, coloque los seis archivos CSV dentro de la subcarpeta `datos` y cree un archivo vacío llamado `lab01.py` en la raíz. Todo el código de este laboratorio se escribe en ese archivo.

Paso 0.4. Verificar que los datos se leen

Escriba en `lab01.py` el siguiente código y ejecútelos desde la carpeta `lab01_<apellido>`:

```
import pandas as pd

RUTA = "datos/"
prueba = pd.read_csv(RUTA + "tiendas.csv")
print(prueba)
```

Ejecute con el comando `python lab01.py`. Debe ver en pantalla las 13 filas del maestro de tiendas. Si obtiene el error `FileNotFoundError`, revise que está ejecutando el programa desde la carpeta correcta y que los archivos están dentro de `datos/`.

Punto de control 0

El maestro de tiendas se imprime con 13 filas y 6 columnas. Doce corresponden a locales físicos y una al canal en línea, con código WEB y cero metros cuadrados de sala. Anote esta particularidad: tendrá consecuencias cuando calcule indicadores por metro cuadrado.

Parte 1. Carga y reconocimiento de las fuentes

Tiempo estimado: 10 minutos.

Paso 1.1. Cargar los seis archivos

Reemplace el contenido de prueba de `lab01.py` por el siguiente bloque. Es la base sobre la que trabajará el resto del laboratorio.

```
import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", 30)

RUTA = "datos/"

ventas = pd.read_csv(RUTA + "ventas_pos_202608.csv")
comprobantes = pd.read_csv(RUTA + "comprobantes_202608.csv")
despachos = pd.read_csv(RUTA + "despachos_202608.csv")
productos = pd.read_csv(RUTA + "productos.csv")
tiendas = pd.read_csv(RUTA + "tiendas.csv")
mensual = pd.read_csv(RUTA + "ventas_mensual_2025_2026.csv")

fuentes = {
    "ventas": ventas, "comprobantes": comprobantes,
```

```
"despachos": despachos, "productos": productos,  
"tiendas": tiendas, "mensual": mensual,  
}  
  
for nombre, df in fuentes.items():  
    print("%-14s filas=%6d columnas=%2d" % (nombre, df.shape[0], df.shape[1]))
```

Paso 1.2. Inspeccionar tipos y primeras filas

Agregue el siguiente bloque para revisar la estructura de la fuente principal:

```
print(ventas.dtypes)  
print(ventas.head(10))
```

Observe con atención la columna `fecha_pedido`. A pesar de que el diccionario de datos la declara como `fecha`, pandas la ha leído como texto (object). Anote esta observación: en la Parte 2 descubrirá el motivo.

Paso 1.3. Registrar el reconocimiento

Complete en su informe la tabla siguiente. La primera fila se entrega resuelta como ejemplo.

Archivo	Filas	Columnas	Llave primaria	Se relaciona con
ventas_pos_202608	34 912	10	id_linea	productos (sku), tiendas (codigo_tienda)
comprobantes_202608				
despachos_202608				
productos				
tiendas				
ventas_mensual_2025_2026				

Tabla 3. Reconocimiento de fuentes (a completar por el estudiante).

Punto de control 1

El archivo de ventas debe cargar 34 912 filas y 10 columnas. El de comprobantes, 12 670 filas. Si sus cifras difieren, verifique que no abrió y volvió a guardar los archivos con Excel: hacerlo altera los formatos de fecha y de número, y es por sí mismo una excelente demostración de por qué el análisis no se hace sobre archivos manipulados manualmente.

Parte 2. Perfilado y diagnóstico de calidad

Tiempo estimado: 20 minutos. Esta es la parte más importante del laboratorio.

En un proyecto real, entre el 60% y el 80% del esfuerzo se consume aquí. Cada paso siguiente detecta un problema distinto. Para cada uno debe registrar cuántas filas afecta y qué decisión toma al respecto.

Paso 2.1. Valores nulos

```
print("--- Nulos por columna (ventas) ---")  
print(ventas.isna().sum())
```

Encontrará valores faltantes en `cantidad` y en `precio_unitario`. Sin esos dos datos la línea no se puede valorizar, de modo que no hay forma de recuperarla: deberá descartarla y dejar constancia de cuántas filas perdió.

Paso 2.2. Filas duplicadas

```
print("Filas duplicadas exactas:", ventas.duplicated().sum())  
print(ventas[ventas.duplicated(keep=False)].sort_values("id_linea").head(6))
```

Note que las filas duplicadas comparten incluso el `id_linea`, que según el diccionario debería ser único. Es un caso típico de reproceso de carga: el mismo lote se cargó dos veces. Si no se eliminan, toda venta calculada quedará inflada.

Paso 2.3. Formatos de fecha inconsistentes

```
print(ventas["fecha_pedido"].head(3).tolist())
con_barra = ventas["fecha_pedido"].str.contains("/").sum()
print("Fechas en formato dd/mm/aaaa:", con_barra)
```

La columna mezcla dos formatos: la mayoría de registros usa `aaaa-mm-dd` y una fracción usa `dd/mm/aaaa`. Esto ocurre porque los sistemas POS de las tiendas no fueron configurados de manera uniforme. Convertir con un solo formato produciría fechas nulas de forma silenciosa, que es la peor clase de error: el programa no falla, simplemente pierde datos sin avisar.

Paso 2.4. Códigos de tienda inconsistentes

```
print(sorted(ventas["codigo_tienda"].unique()))
```

Verá códigos como `T03` y `t03` conviviendo. Si agrupa por esta columna sin normalizarla, cada tienda aparecerá dividida en dos filas del reporte y ninguna mostrará su venta real.

Paso 2.5. Cantidades negativas y precios atípicos

```
print("Cantidades negativas:", (ventas["cantidad"] < 0).sum())
print(ventas["precio_unitario"].describe())
print("Precios sobre 900:", (ventas["precio_unitario"] > 900).sum())
```

Las cantidades negativas corresponden a devoluciones que el POS registró restando en el mismo archivo de ventas, en lugar de emitir una nota de crédito. Los precios extremos son errores de digitación: el operador ingresó el monto sin el separador decimal, multiplicando el valor por cien.

Observe la distribución: la mediana del precio unitario está alrededor de S/ 50 y el percentil 99 alrededor de S/ 148, mientras que los valores atípicos superan los S/ 1 000. Existe un salto limpio entre ambos grupos, lo que permite fijar un umbral defendible.

Paso 2.6. Integridad referencial

```
huerfanos = ~ventas["sku"].isin(productos["sku"])
print("Lineas con SKU inexistente:", huerfanos.sum())
print("SKU distintos no catalogados:", ventas.loc[huerfanos, "sku"].nunique())

sin_pedido = ~comprobantes["id_pedido"].isin(ventas["id_pedido"])
print("Comprobantes sin pedido asociado:", sin_pedido.sum())
```

Son violaciones de integridad referencial: existen ventas de productos que no están en el catálogo y comprobantes emitidos contra pedidos que no existen en el detalle. En una base de datos relacional bien diseñada esto sería imposible, pero aquí los datos provienen de sistemas distintos que no comparten llaves foráneas. Es exactamente el problema que resuelve la capa de integración de la arquitectura vista en la sesión teórica.

Paso 2.7. Categorías no normalizadas

```
print(sorted(productos["categoria"].unique()))
print("Valores distintos:", productos["categoria"].nunique())
```

El catálogo declara seis categorías de negocio, pero la columna contiene veintiún valores distintos por diferencias de mayúsculas, espacios sobrantes y uso de singular en lugar de plural. Un reporte por categoría construido sobre esta columna sería inservible.

Paso 2.8. Aplicar la limpieza

Aplice ahora las reglas de limpieza. El bloque siguiente imprime el efecto de cada regla, que es lo que debe registrar en su informe. Nunca limpie datos sin dejar trazabilidad de cuántos registros afectó cada decisión.

```
v = ventas.copy()
print("Filas iniciales:", len(v))

# R1. Eliminar duplicados exactos
v = v.drop_duplicates()
print("R1 sin duplicados      :", len(v))

# R2. Normalizar el codigo de tienda
v["codigo_tienda"] = v["codigo_tienda"].str.upper().str.strip()

# R3. Parsear la fecha admitiendo los dos formatos presentes
f1 = pd.to_datetime(v["fecha_pedido"], format="%Y-%m-%d", errors="coerce")
f2 = pd.to_datetime(v["fecha_pedido"], format="%d/%m/%Y", errors="coerce")
v["fecha_pedido"] = f1.fillna(f2)
print("R3 fechas sin convertir :", v["fecha_pedido"].isna().sum())

# R4. Descartar lineas sin cantidad o sin precio
v = v.dropna(subset=["cantidad", "precio_unitario"])
print("R4 sin nulos            :", len(v))

# R5. Descartar cantidades negativas (se tratan aparte como devoluciones)
v = v[v["cantidad"] > 0]
print("R5 sin negativos       :", len(v))

# R6. Descartar precios atipicos: mas de 20 veces la mediana
limite = 20 * v["precio_unitario"].median()
print("R6 umbral de precio    : %.2f" % limite)
v = v[v["precio_unitario"] <= limite]
print("R6 sin atipicos        :", len(v))

# R7. Calcular el valor de cada linea (con IGV)
v["valor_linea"] = v["cantidad"] * v["precio_unitario"] * (1 - v["descuento_pct"])

# R8. Normalizar la categoria en el catalogo de productos
prod = productos.copy()
prod["categoria"] = (prod["categoria"].str.strip().str.upper()
                    .replace({"PINTURA": "PINTURAS"}))
print("R8 categorias distintas :", prod["categoria"].nunique())
```

Punto de control 2

La secuencia de filas debe ser: 34 912 iniciales, 34 757 tras R1, 34 472 tras R4, 34 438 tras R5 y 34 429 tras R6. El umbral de precio de R6 debe salir en 993.10 aproximadamente. Tras R3 no debe quedar ninguna fecha sin convertir, y tras R8 deben quedar exactamente 6 categorías.

Si sus números difieren, revise el orden de las reglas. El orden importa: eliminar duplicados antes o después de descartar nulos produce resultados distintos, y debe poder explicar por qué.

Paso 2.9. Registrar el diagnóstico

Complete la tabla siguiente en su informe. Es el principal entregable de esta parte.

Problema detectado	Filas afectadas	Decisión tomada	Justificación
Filas duplicadas exactas	155	Eliminar	Reproceso de carga; inflarían toda venta calculada

Nulos en cantidad o precio			
Dos formatos de fecha			
Códigos de tienda en minúscula			
Cantidades negativas			
Precios atípicos			
SKU no catalogados			
Comprobantes sin pedido			
Categorías no normalizadas			
Precio de lista faltante			

Tabla 4. Registro de diagnóstico y decisiones de limpieza (a completar por el estudiante).

Parte 3. Reproducir el problema de las tres cifras

Tiempo estimado: 15 minutos.

Ahora dispone de datos limpios. Va a calcular la venta de agosto de 2026 tal como la calcula cada una de las tres áreas, siguiendo literalmente la definición que cada una declaró en la reunión.

Paso 3.1. La cifra del área comercial

Comercial define la venta como el total de pedidos registrados en el mes, con IGV incluido y sin descontar los pedidos que luego fueron anulados.

```
comercial = v["valor_linea"].sum()
print("COMERCIAL : S/ %15s" % format(comercial, ",.2f"))
```

Paso 3.2. La cifra del área de finanzas

Finanzas define la venta como el total de comprobantes electrónicos emitidos, sin IGV y neto de notas de crédito. Se consideran únicamente los comprobantes que tienen un pedido asociado en el detalle.

```
IGV = 0.18

comp = comprobantes[comprobantes["id_pedido"].isin(v["id_pedido"])]
emitidos = comp[comp["tipo_comprobante"] != "NOTA_CREDITO"]["monto_total"].sum()
notas_cr = comp[comp["tipo_comprobante"] == "NOTA_CREDITO"]["monto_total"].sum()

finanzas = (emitidos - notas_cr) / (1 + IGV)
print("FINANZAS : S/ %15s" % format(finanzas, ",.2f"))
print("    emitidos : %s" % format(emitidos, ",.2f"))
print("    notas cr. : %s" % format(notas_cr, ",.2f"))
```

Paso 3.3. La cifra del área de logística

Logística define la venta como el valor de las guías de remisión efectivamente despachadas en el mes, valorizadas a precio de lista.

```
desp = despachos[despachos["estado_despacho"] == "DESPACHADO"]
logistica = desp["valor_lista"].sum()
print("LOGISTICA : S/ %15s" % format(logistica, ",.2f"))
```

Paso 3.4. Comparar y cuantificar la brecha

```
cifras = {"Comercial": comercial, "Finanzas": finanzas, "Logistica": logistica}
mayor, menor = max(cifras.values()), min(cifras.values())

print()
for area, valor in cifras.items():
    print("%-10s S/ %15s" % (area, format(valor, ",.2f")))
print("Brecha absoluta : S/ %s" % format(mayor - menor, ",.2f"))
print("Brecha relativa : %.1f%% sobre la cifra menor" % (100 * (mayor / menor - 1)))
```

Punto de control 3

Debe obtener aproximadamente: Comercial S/ 4 780 791, Finanzas S/ 3 884 652 y Logística S/ 4 150 000. La brecha entre la mayor y la menor es de unos S/ 896 000, equivalente al 23% de la cifra menor.

Compare con la sesión teórica: son las mismas tres cifras del caso. Acaba de reproducirlas a partir de los datos, no de una diapositiva.

Paso 3.5. Análisis (responder en el informe)

1. Descomponga la diferencia entre Comercial y Finanzas. ¿Cuánto de la brecha se explica por el IGV, cuánto por los pedidos anulados y cuánto por las notas de crédito? Demuéstrelo numéricamente.
2. ¿Por qué la cifra de Logística no es simplemente un subconjunto de la de Comercial? Identifique al menos dos causas distintas presentes en los datos.
3. ¿Alguna de las tres áreas está calculando mal? Argumente su respuesta.
4. Si el gerente general tuviera que elegir una sola de las tres cifras para su reporte al directorio, ¿cuál recomendaría usted y por qué?

Parte 4. Definición única y panel de indicadores

Tiempo estimado: 15 minutos.

Paso 4.1. Adoptar una definición única

La empresa necesita una sola definición de venta neta, documentada y calculable de una única manera. Se adopta la siguiente, que será la base del panel:

Definición corporativa de VENTA NETA

Valor de las líneas pertenecientes a pedidos NO anulados, calculado como cantidad × precio unitario × (1 - descuento), expresado SIN IGV y neto de las notas de crédito emitidas en el periodo. El reconocimiento se hace en la fecha del pedido.

Universo: pedidos con fecha entre el 01 y el 31 de agosto de 2026, de todos los canales y locales.

Exclusiones: pedidos anulados, líneas sin cantidad o sin precio, y líneas con precio atípico según la regla R6.

Dueño de la definición: Gerencia de Administración y Finanzas.

```
registrados = v[v["estado_pedido"] == "REGISTRADO"]
venta_neta = (registrados["valor_linea"].sum() - notas_cr) / (1 + IGV)
print("VENTA NETA AGOSTO 2026 : S/ %s" % format(venta_neta, ",.2f"))
```

Paso 4.2. Calcular el panel de indicadores

Construya los seis indicadores siguientes sobre la base de la definición única.

```
n_pedidos = registrados["id_pedido"].nunique()
n_pedidos_t = v["id_pedido"].nunique()
unidades = registrados["cantidad"].sum()
```

```
ticket = venta_neta / n_pedidos
lineas_ped = len(registrados) / n_pedidos
tasa_anul = 100 * (1 - n_pedidos / n_pedidos_t)
digital = registrados[registrados["canal"] == "DIGITAL"]
part_digital = 100 * (digital["valor_linea"].sum() / (1 + IGV)) / venta_neta

print("1. Venta neta del mes : S/ %s" % format(venta_neta, ",.2f"))
print("2. Numero de pedidos : %d" % n_pedidos)
print("3. Ticket promedio (s/IGV) : S/ %.2f" % ticket)
print("4. Unidades vendidas : %d" % unidades)
print("5. Líneas por pedido : %.2f" % lineas_ped)
print("6. Tasa de anulación : %.2f %" % tasa_anul)
print("7. Participación del digital : %.2f %" % part_digital)
```

Punto de control 4

Venta neta S/ 3 855 828,30 aproximadamente. Pedidos registrados: 12 349. Ticket promedio: S/ 312,24. Unidades: 88 238. Líneas por pedido: 2,69. Tasa de anulación: 3,83%. Participación del canal digital: 6,58%.

Paso 4.3. Desagregar por tienda y por categoría

```
# Ranking de locales por venta neta
por_tienda = (registrados.groupby("codigo_tienda")["valor_linea"].sum()
              .div(1 + IGV).sort_values(ascending=False).round(2))
por_tienda = por_tienda.rename("venta_neta").reset_index()
por_tienda = por_tienda.merge(tiendas, on="codigo_tienda", how="left")
print(por_tienda[["codigo_tienda", "nombre_tienda", "ciudad", "venta_neta"]].head(5))

# Venta neta por categoria (requiere el catalogo normalizado)
con_cat = registrados.merge(prod[["sku", "categoria"]], on="sku", how="left")
por_cat = (con_cat.groupby("categoria", dropna=False)["valor_linea"].sum()
          .div(1 + IGV).sort_values(ascending=False).round(2))
print(por_cat)
```

En el resultado por categoría aparecerá un grupo con valor nulo (NaN). Corresponde a las líneas cuyo SKU no existe en el catálogo, detectadas en el paso 2.6. No lo elimine ni lo ignore: reportarlo es parte del diagnóstico, porque representa venta real que la empresa no puede atribuir a ninguna categoría.

Paso 4.4. Exportar los resultados

```
import os
os.makedirs("salidas", exist_ok=True)

por_tienda.to_csv("salidas/kpi_por_tienda.csv", index=False, encoding="utf-8")
por_cat.to_csv("salidas/kpi_por_categoria.csv", encoding="utf-8")
print("Archivos exportados en la carpeta salidas/")
```

Parte 5. Clasificación de los indicadores

Tiempo estimado: 10 minutos.

Un indicador no vale por sí mismo sino por la decisión que habilita. Clasifique cada uno de los indicadores calculados según el nivel de decisión de Anthony visto en la sesión teórica, y verifíquelo contra los seis criterios de calidad.

Indicador	Valor obtenido	Nivel de decisión	¿Es un KPI? Justifique	Acción si se desvía
Venta neta del mes		Estratégico	Sí. Tiene meta y dueño.	Revisión del plan comercial
Ticket promedio				
Tasa de anulación				

Participación del canal digital				
Líneas por pedido				
Venta por local				

Tabla 5. Clasificación de indicadores (a completar por el estudiante).

Recuerde la prueba práctica vista en clase: si el indicador se desvía de su meta y nadie en la organización tiene que hacer algo distinto como consecuencia, entonces no es un KPI. Al menos uno de los seis indicadores de la tabla no supera esta prueba. Identifíquelo y explique por qué.

6. Entregable del laboratorio

El trabajo es estrictamente individual. Se entrega a través del Aula Virtual, en un único archivo comprimido con el nombre lab01_<apellido>_<codigo>.zip, hasta 48 horas después de la sesión.

6.1. Contenido del archivo

- lab01.py — el código completo, ejecutable sin modificaciones desde la raíz de la carpeta, comentado y ordenado por partes.
- informe_lab01.pdf — informe de máximo cuatro páginas con las tablas 3, 4 y 5 completas, las respuestas del paso 3.5 y una conclusión de media página.
- salidas/ — los archivos CSV generados por su código.
- No incluya la carpeta datos/ en el archivo comprimido.

6.2. Criterios de evaluación

Criterio	Peso	Qué se evalúa
Código funcional y reproducible	25%	El programa se ejecuta sin errores y reproduce los puntos de control de la guía
Diagnóstico de calidad (Tabla 4)	25%	Los diez problemas identificados, cuantificados y con decisión justificada
Análisis de las tres cifras	20%	La descomposición numérica de la brecha es correcta y está bien argumentada
Panel de indicadores y clasificación	20%	Indicadores correctos, bien clasificados y evaluados contra los seis criterios
Claridad del informe	10%	Redacción, orden y capacidad de sustentar una recomendación

Tabla 6. Rúbrica del Laboratorio 1.

Sobre el uso de asistentes de inteligencia artificial

Está permitido y es esperable que utilice asistentes de IA durante el laboratorio. Debe declararlo en una línea al final del informe, indicando en qué pasos lo empleó.

La condición es que usted responde por todo lo entregado. En la clase siguiente se le puede pedir que explique cualquier línea de su código o que justifique cualquier decisión de limpieza. No poder hacerlo invalida la entrega, independientemente de que el resultado sea correcto.

7. Actividad propuesta: la decisión de expansión

Tiempo estimado: 40 minutos. Trabajo individual. Se inicia en clase y se adjunta al mismo entregable.

7.1. Situación

El directorio de Comercial Los Andes aprobó abrir cuatro locales nuevos. La gerencia general debe decidir en qué ciudad concentrarlos y le solicita a usted un sustento con datos, no con impresiones. Dispone del archivo `ventas_mensual_2025_2026.csv`, que contiene el agregado mensual por tienda y categoría desde enero de 2025 hasta agosto de 2026.

Este archivo ya viene consolidado y sin los problemas de calidad del detalle transaccional, de modo que puede concentrarse en el análisis.

7.2. Columnas disponibles

Columna	Descripción
<code>periodo</code>	Mes en formato aaaa-mm
<code>codigo_tienda</code>	Código del local; se relaciona con <code>tiendas.csv</code>
<code>categoria</code>	Categoría de producto, ya normalizada
<code>unidades</code>	Unidades vendidas en el mes
<code>venta_neta</code>	Venta neta del mes en soles, sin IGV
<code>costo_mercaderia</code>	Costo de la mercadería vendida, en soles
<code>num_transacciones</code>	Número de transacciones del mes
<code>num_clientes</code>	Número de clientes distintos del mes

Tabla 7. Estructura del archivo de agregado mensual.

7.3. Tareas

Escriba su código en un segundo archivo llamado `actividad01.py`.

Tarea 1 — Construir tres indicadores comparables entre ciudades (15 minutos)

Calcule, para cada ciudad, al menos los tres indicadores siguientes. Debe usar el archivo de `tiendas` para incorporar la ciudad y los metros cuadrados de sala.

- Margen bruto porcentual: $(venta_neta - costo_mercaderia) / venta_neta$. Indica qué tan rentable es la mezcla de productos que se vende en cada plaza.
- Venta neta por metro cuadrado de sala, en el acumulado de los últimos doce meses. Es el indicador estándar de productividad del retail. Atención: el canal en línea tiene cero metros cuadrados y debe excluirse de este cálculo.
- Crecimiento interanual: variación de la venta neta del periodo enero–agosto 2026 respecto del mismo periodo de 2025.

Sugerencia de punto de partida:

```
import pandas as pd

mensual = pd.read_csv("datos/ventas_mensual_2025_2026.csv")
tiendas = pd.read_csv("datos/tiendas.csv")

m = mensual.merge(tiendas, on="codigo_tienda", how="left")
m["anio"] = m["periodo"].str[:4].astype(int)
m["mes"] = m["periodo"].str[5:7].astype(int)

# A partir de aquí construya sus indicadores por ciudad.
```

Tarea 2 — Verificar los indicadores (10 minutos)

Someta cada uno de sus tres indicadores a los seis criterios de calidad vistos en la sesión teórica: específico, medible y reproducible, alineado, accionable, temporal y contextualizado. Registre el resultado en la tabla siguiente y corrija la definición de aquellos que no aprueben.

Indicador	Específico	Medible	Alineado	Accionable	Temporal	Contextualizado

Tabla 8. Verificación de indicadores contra los criterios de calidad.

Tarea 3 — Redactar la recomendación (15 minutos)

Elabore una recomendación de máximo una página dirigida a la gerencia general, que contenga:

- La ciudad recomendada para concentrar la expansión, enunciada en la primera línea.
- Los tres indicadores que sustentan la recomendación, con sus valores para las tres ciudades.
- Al menos un argumento en contra de su propia recomendación. Toda decisión gerencial tiene contrapartida, y omitirla es un defecto de análisis.
- Qué dato que hoy no existe le habría permitido dar una recomendación más sólida. Esta pregunta prepara el trabajo de las semanas 2 a 5.

Criterio de calidad de la recomendación

No se evalúa qué ciudad eligió: los datos admiten más de una respuesta defendible según qué indicador se privilegie. Se evalúa que la recomendación esté sustentada en indicadores correctamente calculados, que reconozca sus propias limitaciones y que sea accionable.

Una recomendación que dice «se sugiere analizar más a fondo» no es una recomendación.

Anexo A. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
FileNotFoundError al leer un CSV	El programa se ejecuta desde una carpeta distinta a la raíz del proyecto	Sitúese en la carpeta lab01_<apellido> antes de ejecutar, o use rutas absolutas de forma temporal
UnicodeDecodeError	El archivo se abrió y guardó con Excel, alterando la codificación	Vuelva a extraer los archivos originales del comprimido; no los abra con Excel
Las tildes se ven como caracteres raros	La consola de Windows no usa UTF-8	Ejecute chcp 65001 en la terminal antes de correr el programa
Los totales salen mucho mayores a los del punto de control	No se eliminaron los duplicados, o se aplicó la limpieza en un orden distinto	Revise que R1 se ejecute antes de cualquier suma
to_datetime devuelve muchos valores nulos	Se está forzando un solo formato de fecha	Aplique los dos formatos y combine con fillna, como en R3
El agrupamiento por tienda devuelve más de 13 grupos	No se normalizó el código de tienda a mayúsculas	Verifique que R2 se ejecutó sobre el DataFrame que está usando
SettingWithCopyWarning	Se modifica un subconjunto de un DataFrame sin copiarlo	Use .copy() al crear el subconjunto, como se hace en el paso 2.8

Anexo B. Equivalencias entre pandas y SQL

Esta tabla conecta las operaciones del laboratorio con el lenguaje que usted ya domina de la asignatura Base de Datos II.

Operación	pandas	SQL
Seleccionar columnas	df[["a", "b"]]	SELECT a, b FROM t
Filtrar filas	df[df["a"] > 10]	WHERE a > 10
Contar filas	len(df)	SELECT COUNT(*) FROM t

Valores distintos	<code>df["a"].nunique()</code>	<code>SELECT COUNT(DISTINCT a) FROM t</code>
Agrupar y sumar	<code>df.groupby("a")["b"].sum()</code>	<code>SELECT a, SUM(b) FROM t GROUP BY a</code>
Ordenar	<code>df.sort_values("b", ascending=False)</code>	<code>ORDER BY b DESC</code>
Combinar tablas	<code>df1.merge(df2, on="k", how="left")</code>	<code>LEFT JOIN ON df1.k = df2.k</code>
Eliminar duplicados	<code>df.drop_duplicates()</code>	<code>SELECT DISTINCT * FROM t</code>
Detectar huérfanos	<code>~df1["k"].isin(df2["k"])</code>	<code>WHERE k NOT IN (SELECT k FROM t2)</code>
Descartar nulos	<code>df.dropna(subset=["a"])</code>	<code>WHERE a IS NOT NULL</code>

Como ejercicio opcional, escriba la consulta SQL que produciría la cifra del área de finanzas del paso 3.2, asumiendo que las tres fuentes estuvieran cargadas como tablas en una base de datos relacional. Compárela con el código pandas equivalente y reflexione sobre cuál resulta más legible para auditar una definición de métrica.